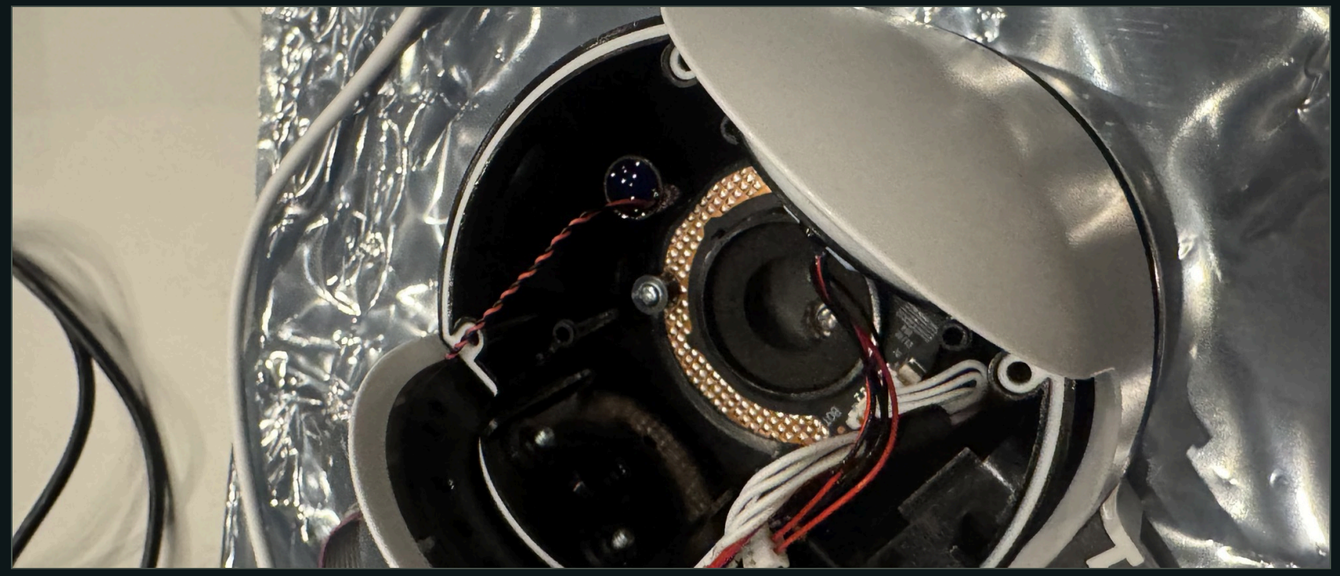
THE CHALLENGE · REAL HARDWARE

It was watching the garden. We looked inside.



TAPO C500 2.0

A camera. A small computer.

Its memory holds software, settings and files.

OUR GOAL

BA26{…}

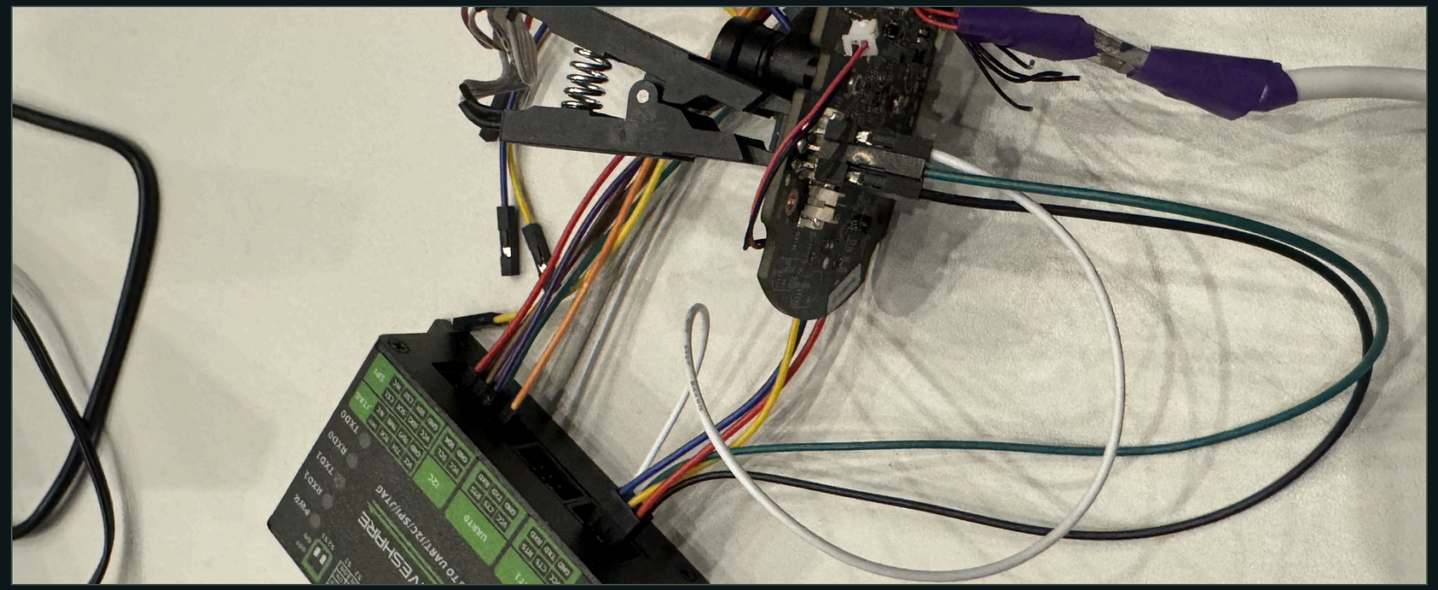
Recover the hidden message.

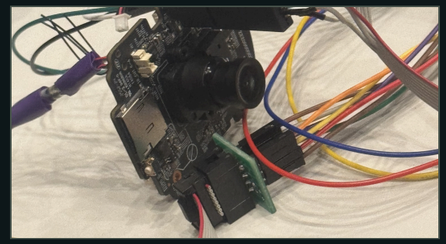
THE TAKEAWAY

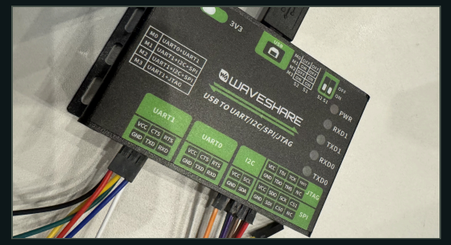
The starting point for analysis: **an 8 MiB memory dump.**

ACQUISITION · COPY THE MEMORY TO STUDY IT

A lot of wires.
One file at the end.







Flash memory

↓ read

Adapter ⟶ computer

↓ copy

`dump · 8 MiB`

THE TAKEAWAY

The investigation continues **inside the recovered file.**

FIRST ATTEMPT · A TEXT SEARCH IS NOT ENOUGH

Ctrl+F found nothing.
We needed a different view.

01 / STORAGE

Compressed files

Take up less space.



→ `files`

Decompress to reach the contents.

02 / MESSAGE

An encrypted flag

Make the text unreadable.

```
9c b6 c8 70 d8 3a…
```

Understand the recipe to recover it.

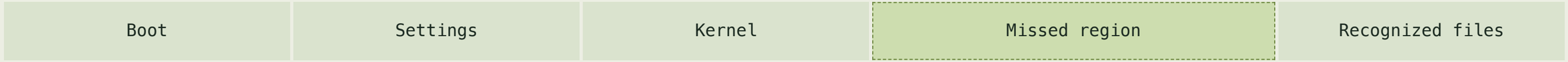
THE TAKEAWAY

This challenge combines file compression and message encryption.

THE BREAKTHROUGH · EXTRACTION HAD MISSED A USEFUL REGION

The book had lost its cover. Not its pages.

DUMP · DIAGRAM NOT TO SCALE



UNRECOGNIZABLE HEADER

?

The first extraction does not see everything.

→

INTACT METADATA

Position	Where to read
Size	How much to read
Fragment	Which block to open

→

RECOVERED FILES

ctf_unlock

flag.enc

Read the right bytes.

THE TAKEAWAY

The tool had missed something. **The memory still had a story to tell.**

THE CLUES · THREE COMPLEMENTARY PIECES

The message was secret.
The instructions were next door.

01

flag.enc

The encrypted message.
35 bytes in the supplied file.

+

02

ctf_unlock

The program that explains how to recover the text.

+

03

devId

The camera's identifier, stored in its settings.

AT STARTUP `devId → temporary key → ctf_unlock + flag.enc → /tmp/flag.txt`

THE TAKEAWAY

Startup connects the clues: **identifier + program + encrypted message.**

THE MECHANISM · REPRODUCE THE RECIPE

Same ingredients.
Exactly the same mask.

`devId · identifier` + `“C500-CTF-v1”` + `counter: 0, then 1`

↓ **SHA-256** One 32-byte hash per counter value.



The message is **35 bytes** : use the first block, then 3 bytes of the second.

THE TAKEAWAY

We compute SHA-256 in the usual direction. **We do not reverse it.**

A SHORT PAUSE · THE AUDIENCE CAN DECRYPT TOO

An operation that can go into reverse.

XOR

Two identical bits → **0**

Two different bits → **1**

A four-bit teaching example.
These are not the actual flag bits.

Message

1 0 1 0

Mask

0 1 1 0

Encrypted

1 1 0 0

Encrypted

1 1 0 0

Same mask

0 1 1 0

Message

1 0 1 0

THE TAKEAWAY

Applying **the same mask twice** restores the original message.

FROM DIAGRAM TO CODE · ONE EXCERPT, THREE ACTIONS

The recipe fits in Python. Every line has a role.

01	`key + b'C500-CTF-v1' + struct.pack('<I', counter)`	Assemble the ingredients. The counter takes 0, then 1.
02	`hashlib.sha256(…).digest()`	Produce a 32-byte block. Join the two blocks end to end.
03	`bytes(a ^ b for a, b in zip(encrypted, stream))`	Combine each encrypted byte with its mask byte.

Excerpts from solve.py, separated for readability; "…" abbreviates the ingredients. The script does not execute ctf_unlock.

THE TAKEAWAY

The script connects the extracted files, identifier and recipe. **The result becomes verifiable.**

THE RESULT · TEXT PRESERVED IN FLAG.TXT

Smile.
The flag is out.

EXACT CONTENTS OF THE SUPPLIED FLAG.TXT FILE

```
BA26{a-R00t-0f-Tru5t-Sh1ps-N0-K3ys}
```

35 characters · the same result as in the PowerPoint.

DOCUMENTED STATUS **Flag accepted.** Submission confirmed by the platform.

THE TAKEAWAY

The secret shared its memory with **its recipe and the data needed to read it.**